

Relatório TP2 – Ordenação de Objetos Móveis com Base na Trajetória

Rodrigo Chaves 24.1.4037 e Lázaro Alves 23.1.4104
Universidade Federal de Ouro Preto

15 de julho de 2025

1 Implementação

Este trabalho tem como objetivo implementar um sistema que analisa e ordena objetos móveis com base em suas trajetórias no plano cartesiano. Cada objeto é representado por uma sequência de pontos, e o programa calcula duas grandezas principais: a distância percorrida (escalar) e o deslocamento (vetorial).

A implementação foi feita em linguagem C e está organizada em múltiplos arquivos, seguindo a convenção de modularização: 'tp.c', 'ordenacao.c/.h', 'objeto.c/.h', e 'ponto.h'.

ponto.h

Define a estrutura e operações para o TAD ponto, essencial para representar as posições dos objetos no plano. **Principais operações:**

- `criaPonto(x, y)`: cria dinamicamente um novo ponto.
- `liberaPonto(p)`: libera a memória associada ao ponto.
- `distanciaEntre(p1, p2)`: calcula a distância euclidiana entre dois pontos usando o teorema de Pitágoras.

```
#ifndef PONTO_H
#define PONTO_H

typedef struct ponto
{
    int x, y;
} Ponto;

#endif
```

objeto.h / objeto.c

Responsável por representar um objeto móvel (identificador + vetor de pontos) e calcular as métricas associadas à sua trajetória:

- **criaObjeto** e **liberaObjeto**: alocação dinâmica.
- **calculaDeslocamento**: aplica o teorema de Pitágoras sobre os pontos inicial e final.
- **calculaDistanciaPercorrida**: soma das distâncias entre os pontos consecutivos.

```
#ifndef OBJETO_H
#define OBJETO_H
#include <stdio.h>
#include "ponto.h"

typedef struct objeto
{
    char nome[5];
    Ponto* ponto;
    float deslocamento;
    float trajetoria;
} Objeto;

Objeto* alocaObjetos(int numObjetos, int numPontos);
void destroiObjetos(Objeto* objetos, int numObjetos);

#endif
```

```
#include <stdio.h>
#include <stdlib.h>
#include "objeto.h"
#include "ponto.h"

Objeto* alocaObjetos(int numObjetos, int numPontos) {
    Objeto* objetos = (Objeto*)malloc(sizeof(Objeto) * numObjetos
    );
    if (!objetos)
        return NULL;

    for (int i = 0; i < numObjetos; i++) {
        objetos[i].ponto = (Ponto*)malloc(sizeof(Ponto) *
        numPontos);
        if (objetos[i].ponto == NULL) {
            for (int j = 0; j < i; j++)
                free(objetos[j].ponto);

            free(objetos);
            return NULL;
        }
    }
}
```

```

        return objetos;
    }

void destroiObjetos(Objeto* objetos, int numObjetos) {
    for (int i = 0; i < numObjetos; i++)
        free(objetos[i].ponto);
    free(objetos);
}

```

Módulo de Ordenação (ordenacao.h / ordenacao.c)

Este módulo realiza a ordenação dos objetos móveis. O critério de ordenação é triplo:

1. Maior distância percorrida primeiro.
2. Em caso de empate, menor deslocamento.
3. Persistindo empate, ordem lexicográfica do nome.

Para isso, utiliza-se a função de comparação e um algoritmo estável de ordenação (neste caso, um Merge Sort adaptado).

Funções principais:

- `mergeSort(objetos, n)`: ordena o vetor de objetos com base no critério múltiplo.
- `compara(obj1, obj2)`: função auxiliar que define a prioridade de ordenação entre dois objetos.

```

#ifndef ORDENACAO_H
#define ORDENACAO_H

#include <stdio.h>
#include <stdbool.h>
#include "ponto.h"
#include "objeto.h"

float calcDeslocamento(Ponto* inicio, Ponto* fim);
float calcTrajetoria(Objeto* objeto, int n);

void mergerec(Objeto* objetos, int l, int r);
double round2(double valor);
void merge(Objeto* objetos, int l, int m, int r);
void MergeSort(Objeto* objetos, int n);

void imprimeOrdenado(Objeto* objeto, int n);

#endif

```

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <math.h>
#include <string.h>
#include "objeto.h"
#include "ordenacao.h"
#include "ponto.h"

typedef enum {
    MENOR=1,
    IGUAL,
    MAIOR
} COMPARACAO;

float calcDeslocamento(Ponto* inicio, Ponto* fim) {
    float dx = fim->x - inicio->x;
    float dy = fim->y - inicio->y;
    float aux = sqrt((dx * dx) + (dy * dy));
    return aux;
}

float calcTrajetoria(Objeto* objeto, int n) {

    double soma = 0;
    for (int i = 0; i < n - 1; i++)
        soma += calcDeslocamento(&objeto->ponto[i], &objeto->
            ponto[i + 1]);

    return soma;
}

COMPARACAO ItemCompara(float i1, float i2) {

    if (i1 > i2)
        return MENOR;
    else if (i1 < i2)
        return MAIOR;
    return IGUAL;
}

COMPARACAO ItemComparaV2(char* nome1, char* nome2) {
    if (strcmp(nome1, nome2) < 0)
        return MENOR;
    else if (strcmp(nome1, nome2) > 0)
        return MAIOR;
    return IGUAL;
}

```

```

void MergeSort(Objeto* objetos, int n) {
    mergerec(objetos, 0, n - 1);
}

void mergerec(Objeto* objetos, int l, int r) {
    if (r > l) {
        int m = (l + r) / 2;
        mergerec(objetos, l, m);
        mergerec(objetos, m + 1, r);
        merge(objetos, l, m, r);
    }
}

double round2(double valor) {
    return round(valor * 100) / 100;    // Ex: 3.1415      3.14
}

void merge(Objeto* objetos, int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    Objeto* L = (Objeto*)malloc(n1 * sizeof(Objeto));
    Objeto* R = (Objeto*)malloc(n2 * sizeof(Objeto));

    for (int i = 0; i < n1; i++) L[i] = objetos[l + i];
    for (int j = 0; j < n2; j++) R[j] = objetos[m + 1 + j];

    int i = 0, j = 0, k = l;

    while (i < n1 && j < n2) {
        // Arredonda para 2 casas decimais antes de comparar
        double trajL = round2(L[i].trajetoria);
        double trajR = round2(R[j].trajetoria);
        double deslL = round2(L[i].deslocamento);
        double deslR = round2(R[j].deslocamento);

        // 1 Critério: Trajetória (decrecente)
        if (fabs(trajL - trajR) > 1e-9) { // Pequena tolerância
            para evitar erros de arredondamento
            if (trajL > trajR) {
                objetos[k++] = L[i++];
            } else {
                objetos[k++] = R[j++];
            }
        }

        // 2 Critério: Deslocamento (crescente)
        else if (fabs(deslL - deslR) > 1e-9) {
            if (deslL < deslR) {

```

```

        objetos[k++] = L[i++];
    } else {
        objetos[k++] = R[j++];
    }
}
// 3 Critério: Nome (alfabético)
else {
    if (strcmp(L[i].nome, R[j].nome) < 0) {
        objetos[k++] = L[i++];
    } else {
        objetos[k++] = R[j++];
    }
}

while (i < n1) objetos[k++] = L[i++];
while (j < n2) objetos[k++] = R[j++];

free(L);
free(R);
}

void imprimeOrdenado(Objeto* objetos, int n) {
    for(int i = 0; i < n; i++)
        printf("%s %.2f %.2f\n", objetos[i].nome, objetos[i].
            trajetoria, objetos[i].deslocamento);
}

```

tp.c

Este é o programa principal, responsável por:

- Ler a quantidade de objetos e a quantidade de pontos por objeto.
- Ler os dados das trajetórias e armazená-los nos objetos.
- Calcular distância percorrida e deslocamento de cada objeto.
- Ordenar os objetos e imprimir os resultados no formato especificado.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "ordenacao.h"
#include "objeto.h"
#include "ponto.h"

int main() {

```

```

int numObjetos, totalPontos;
scanf("%d%d", &numObjetos, &totalPontos);

Objeto* objetos = alocaObjetos(numObjetos, totalPontos);
for (int i = 0; i < numObjetos; i++)
{
    scanf("%s", objetos[i].nome);
    for (int j = 0; j < totalPontos; j++)
        scanf("%d%d", &objetos[i].ponto[j].x, &objetos[i].
            ponto[j].y);

    objetos[i].deslocamento = calcDeslocamento(&objetos[i].
        ponto[0], &objetos[i].ponto[totalPontos - 1]);
    objetos[i].trajetoria = calcTrajetoria(&objetos[i],
        totalPontos);
}

MergeSort(objetos, numObjetos);
imprimeOrdenado(objetos, numObjetos);

destroiObjetos(objetos, numObjetos);
return 0;
}

```

2 Impressões Gerais

O trabalho exigiu a aplicação de conceitos fundamentais como modularização, ponteiros, alocação dinâmica e uso de estruturas. Foi interessante observar como a definição adequada de TADs facilitou a implementação e organização do código. Uma das partes mais satisfatórias foi a integração entre os módulos e a correta liberação de memória com auxílio do Valgrind.

3 Análise

O programa apresentou desempenho eficiente para o conjunto de dados utilizado nos testes. Como cada trajetória possui no máximo dezenas de pontos, o tempo de execução permanece praticamente constante. A complexidade da ordenação é $O(n \log n)$, e o custo das funções de distância e deslocamento é linear no número de pontos.

4 Conclusão

Este trabalho reforçou a importância da organização modular de código e o uso correto de estruturas de dados para problemas de análise e ordenação. A principal dificuldade foi garantir a ordenação correta com múltiplos critérios, além de evitar vazamentos de memória.